System Design: What is Scalability?

#1 System Design - Scalability

ASHISH PRATAP SINGH
MAR 04, 2024

As a system grows, the performance starts to **degrade** unless we adapt it to deal v that growth.

Scalability is the property of a system to handle a growing amount of load by **adc resources** to the system.

> A system that can continuously evolve to support a growing amount of work i scalable.

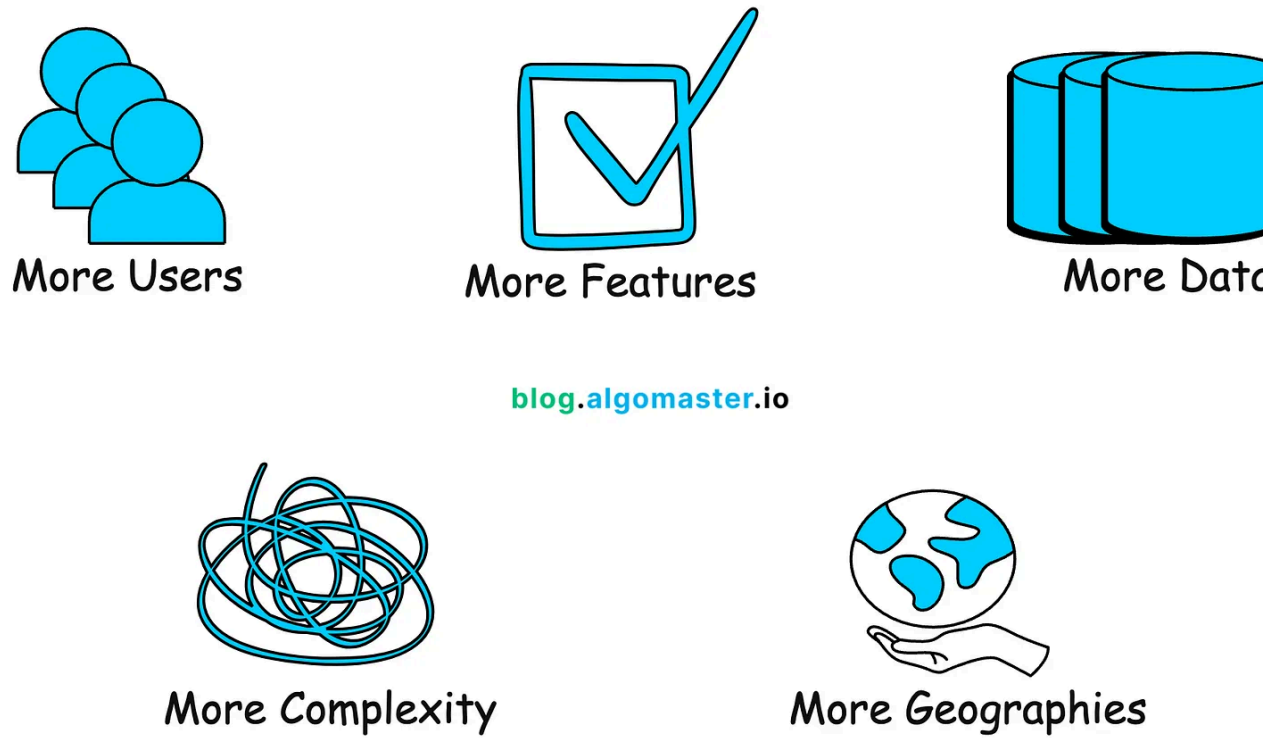
In this article, we will explore different ways a system can grow and common wa make a system scalable.

If you're enjoying this newsletter and want to get even more value, consider becc a [paid subscriber](#).

As a paid subscriber, you'll unlock all **premium articles** and gain full access to al [premium courses](#) on [algomaster.io](#).

How can a System Grow?

A system can grow in several dimensions.



1. Growth in User Base

More users started using the system, leading to increased number of requests.

- **Example:** A social media platform experiencing a surge in new users.

2. Growth in Features

More features were introduced to expand the system's capabilities.

- **Example:** An e-commerce website adding support for a new payment metho

3. Growth in Data Volume

Growth in the amount of data the system stores and manages due to user activity logging.

- **Example:** A video streaming platform like youtube storing more video conte over time.

4. Growth in Complexity

The system's architecture evolves to accommodate new features, scale, or integra resulting in additional components and dependencies.

- **Example:** A system that started as a simple application is broken into smalle independent systems.

5. Growth in Geographic Reach

The system is expanded to serve users in new regions or countries.

- **Example:** An e-commerce company launching websites and distribution in i international markets.

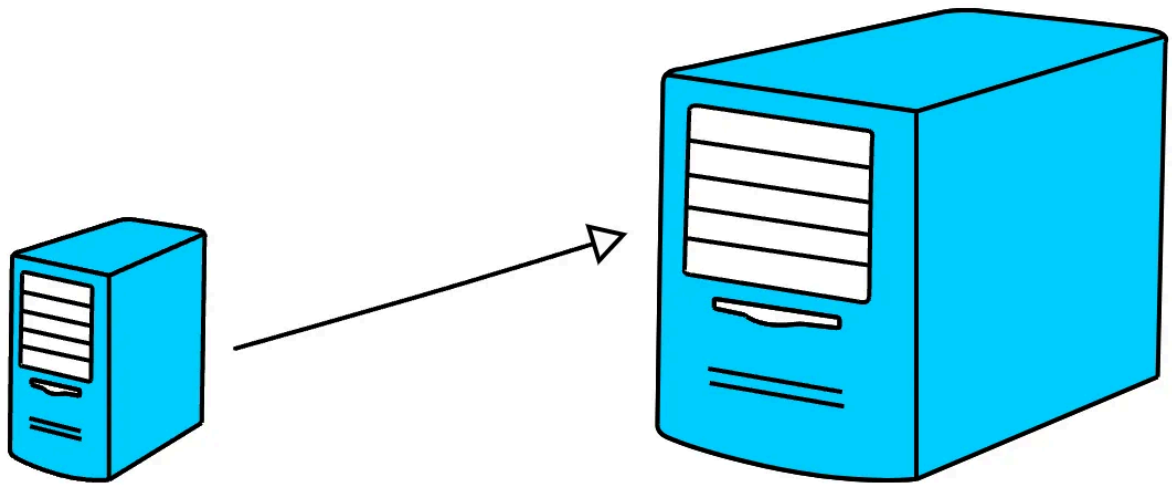
How to Scale a System?

Here are 10 common ways to make a system scalable:

1. Vertical Scaling (Scale up)

This means adding more power to your existing machines by upgrading server w more RAM, faster CPUs, or additional storage.

It's a good approach for simpler architectures but has limitations in how far you go.

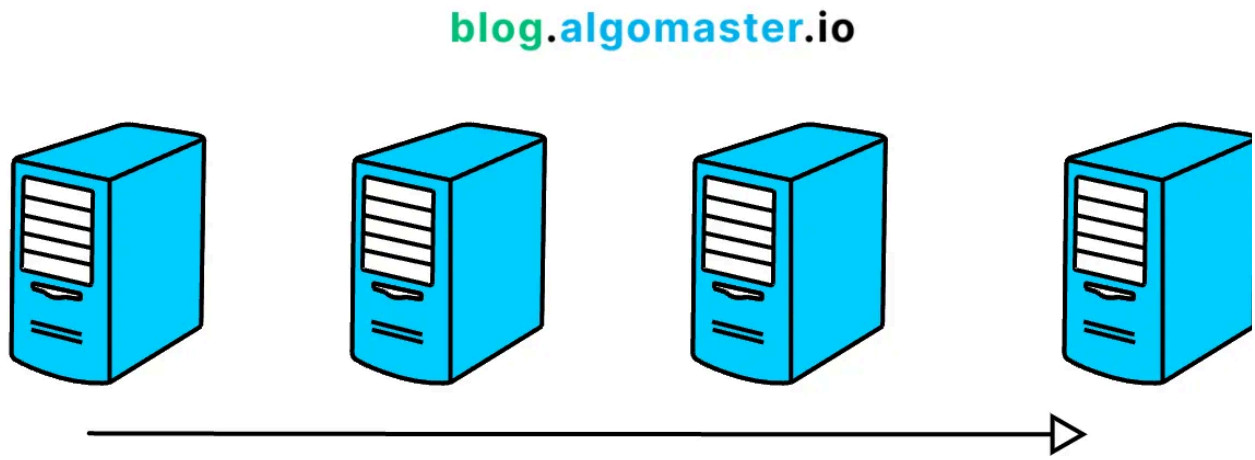


blog.algomaster.io

2. Horizontal Scaling (Scale out)

This means adding more machines to your system to spread the workload across multiple servers.

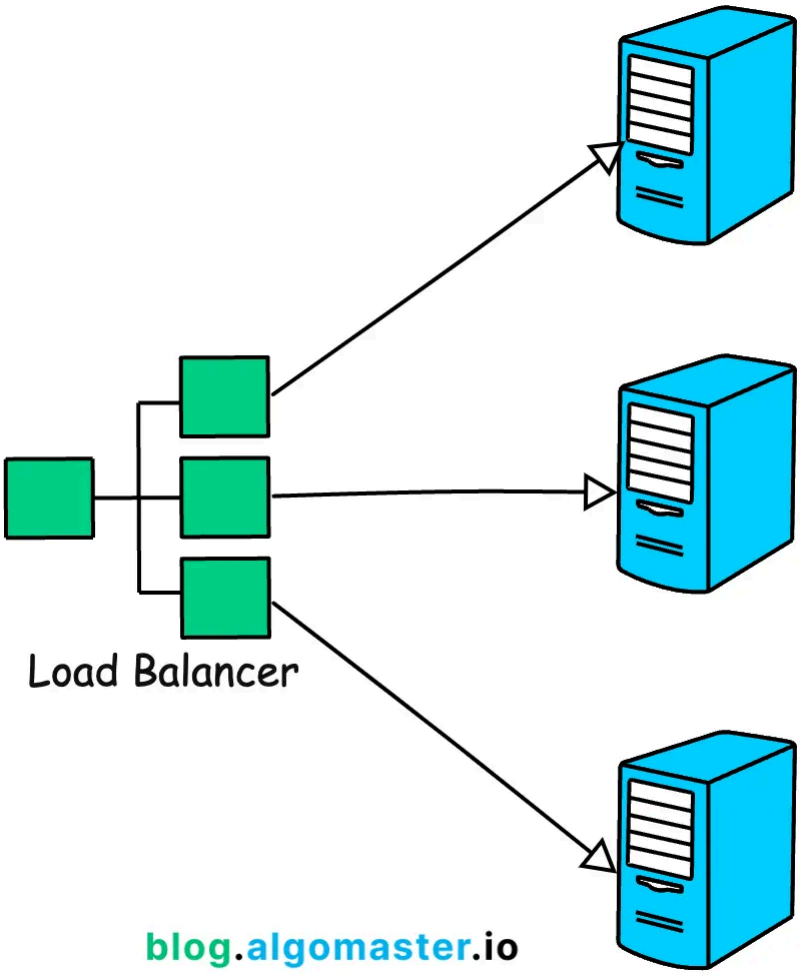
It's often considered the most effective way to scale for large systems.



> **Example:** Netflix uses horizontal scaling for its streaming service, adding mo servers to their clusters to handle the growing number of users and data traffi

3. Load Balancing

Load balancing is the process of distributing traffic across multiple servers to en no single server becomes overwhelmed.

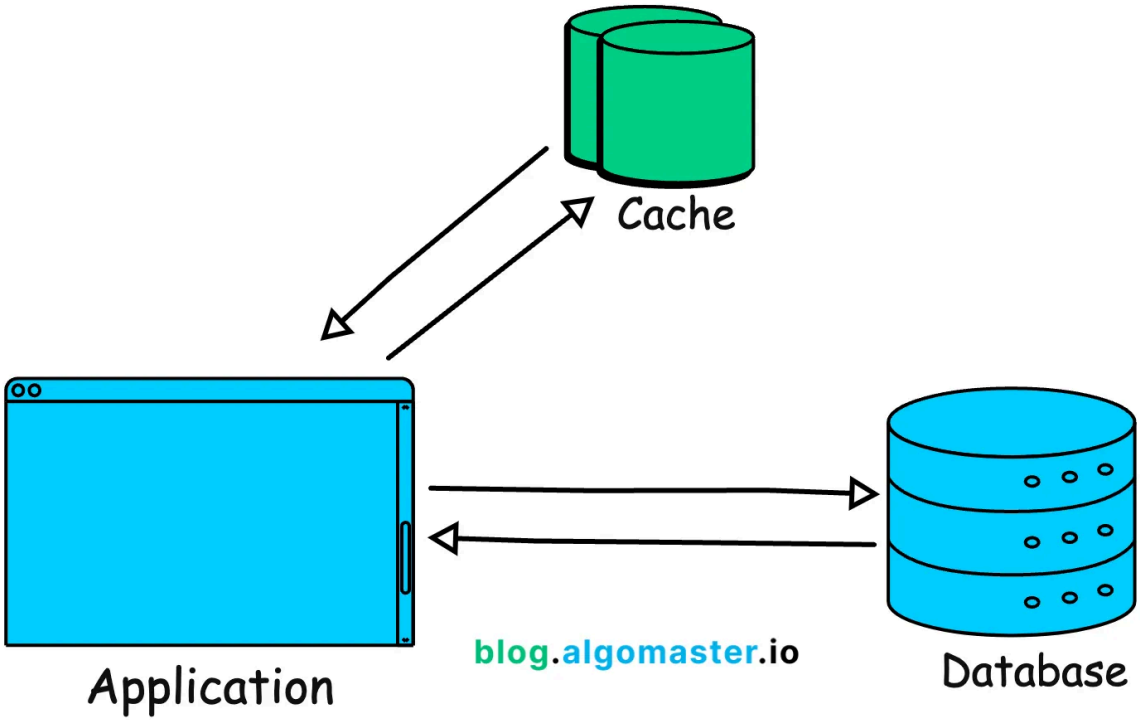


> **Example:** Google employs load balancing extensively across its global infrastructure to distribute search queries and traffic evenly across its massive server farms.

4. Caching

Caching is a technique to store frequently accessed data in-memory (like RAM) t reduce the load on the server or database.

Implementing caching can dramatically improve response times.

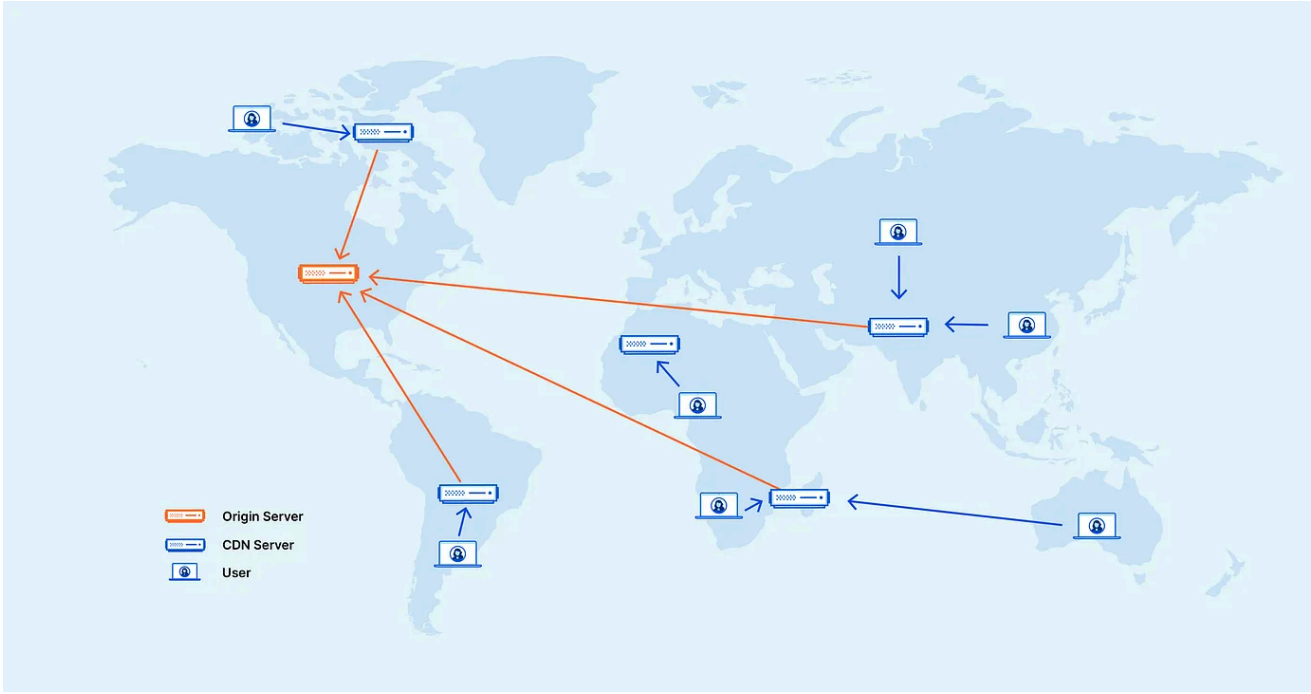


> **Example:** Reddit uses caching to store frequently accessed content like hot pc and comments so that they can be served quickly without querying the databa each time.

5. Content Delivery Networks (CDNs)

CDN distributes static assets (images, videos, etc.) closer to users. This can reduc latency and result in faster load times.

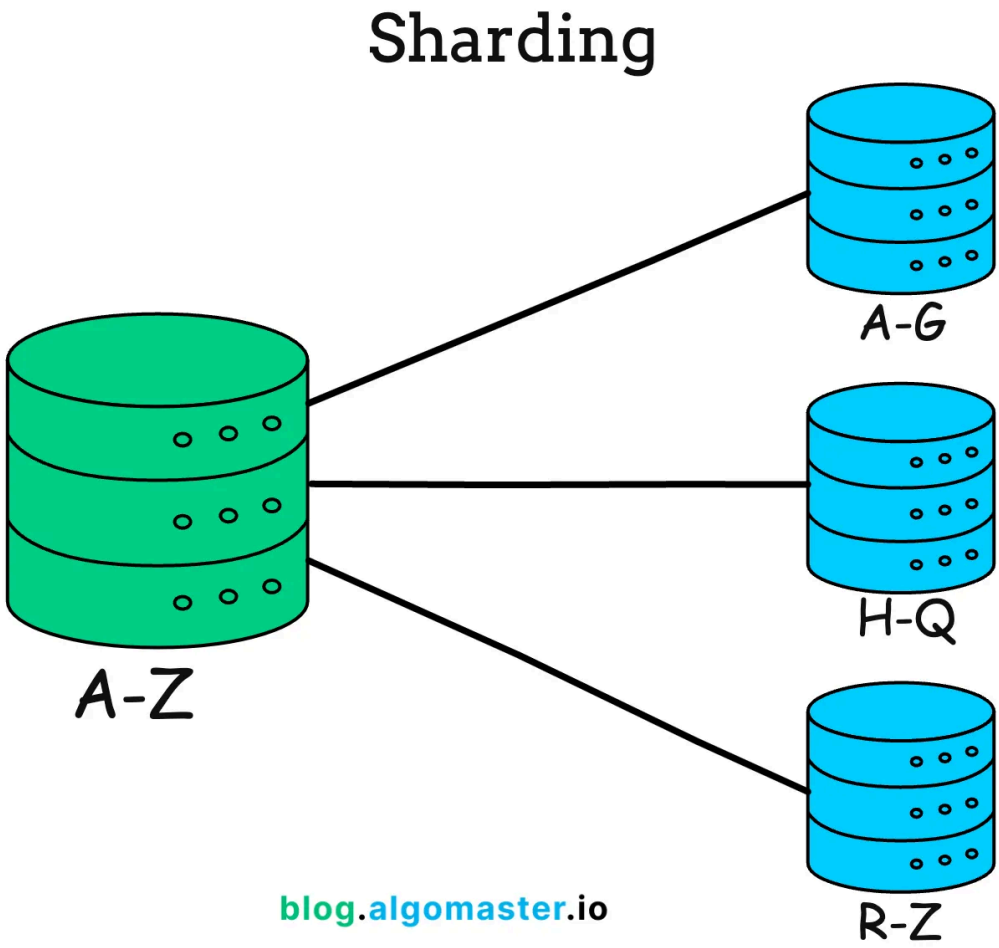
> **Example:** Cloudflare provides CDN services, speeding up website access for u worldwide by caching content in servers located close to users.



Credit: https://www.cloudflare.com/learning/cdn/what-is-a-cdn/

6. Sharding/Partitioning

Partitioning means splitting data or functionality across multiple nodes/servers t distribute workload and avoid bottlenecks.



> **Example:** Amazon DynamoDB uses partitioning to distribute data and traffic NoSQL database service across many servers, ensuring fast performance and scalability.

7. Asynchronous communication

Asynchronous communication means deferring long-running or non-critical tasl background queues or message brokers.

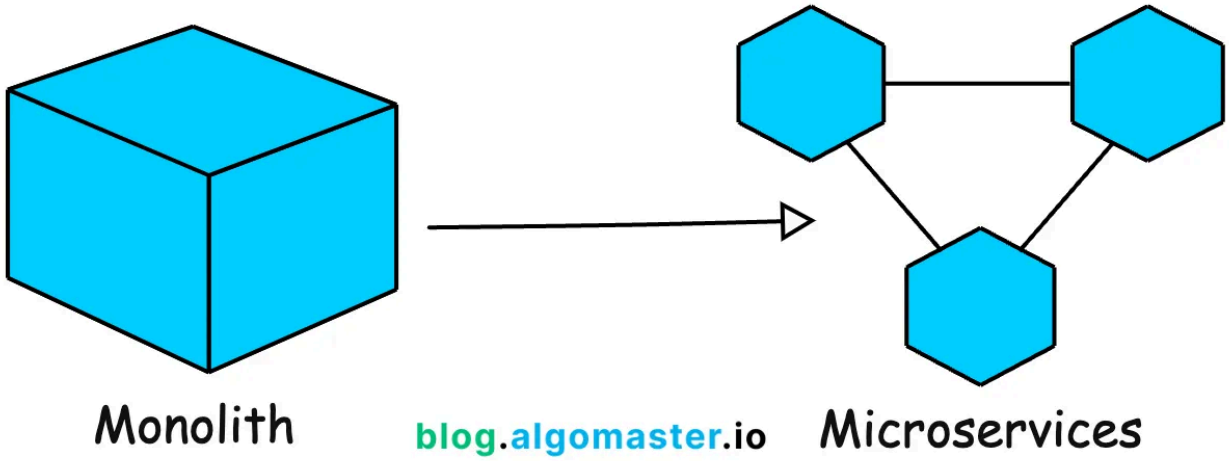
This ensures your main application remains responsive to users.

> **Example:** Slack uses asynchronous communication for messaging. When a me is sent, the sender's interface doesn't freeze; it continues to be responsive whi message is processed and delivered in the background.

8. Microservices Architecture

Micro-services architecture breaks down application into smaller, independent services that can be scaled independently.

This improves resilience and allows teams to work on specific components in pa

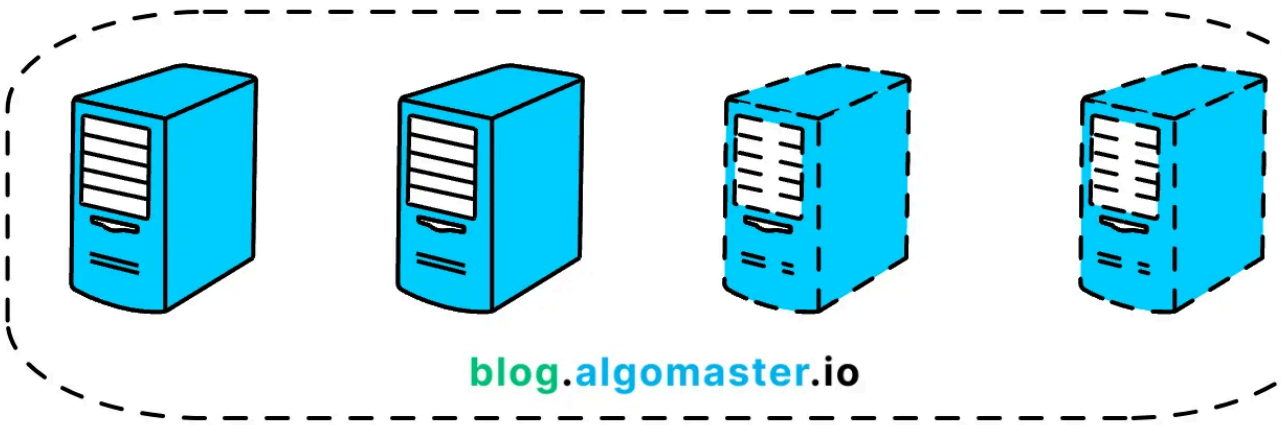


> **Example:** Uber has evolved its architecture into microservices to handle diffe functions like billing, notifications, and ride matching independently, allowin efficient scaling and rapid development.

9. Auto-Scaling

Auto-Scaling means automatically adjusting the number of active servers based current load.

This ensures that the system can handle spikes in traffic without manual interve



> **Example:** AWS Auto Scaling monitors applications and automatically adjusts capacity to maintain steady, predictable performance at the lowest possible cc

10. Multi-region Deployment

Deploy the application in multiple data centers or cloud regions to reduce latenc improve redundancy.

> **Example:** Spotify uses multi-region deployments to ensure their music stream service remains highly available and responsive to users all over the world, regardless of where they are located.

Thank you for reading!

If you found it valuable, hit a like ❤️ and consider subscribing for more such cor every week.

If you have any questions or suggestions, leave a comment.

This post is public so feel free to share it.

P.S. If you're enjoying this newsletter and want to get even more value, consider becoming a **paid subscriber**.

As a paid subscriber, you'll unlock all **premium articles** and gain full access to al **premium courses** on **algomaster.io**.

There are group discounts, gift options, and referral bonuses available.

Checkout my **Youtube channel** for more in-depth content.

Follow me on LinkedIn, X and Medium to stay updated.

Checkout my GitHub repositories for free interview preparation resources.

I hope you have a lovely day!

See you soon,
Ashish

577 Likes · 27 Restacks

Nex

Discussion about this post

Comments Restacks

Write a comment...

Shivam Singh Shivam Singh May 9, 2024

Liked by Ashish Pratap Singh

Wonderful explanation . After reading this article i am feeling confident that if i read and und your article , i will be good in system design.

Thanks a lot bro. Keep Growing.

Waiting for LLD and HLD videos on each topic .

LIKE (6) REPLY

1 reply by Ashish Pratap Singh

Ram Apr 17, 2024

Liked by Ashish Pratap Singh

Liked it. Very good explanation which is need for interview prep.

LIKE (3) REPLY

1 reply by Ashish Pratap Singh

37 more comments...